



Rust Moravia

Why do we all write tests that...

(Select all that apply to your tests)

- ☒ Are hard to maintain
- ☒ Are hard to understand
- ☒ Are hard to reproduce failures
- ☒ Break during refactoring
- ☒ Have lots of duplicated code
- ☒ Create unnecessary stress
- ☒ Other (specify)

Because...

(Select your favourite excuse)

- ☐ It's a natural inevitability, there's no other way
- ☐ We like it that way (if it was hard to write, it should be hard to understand)
- ☐ Our production code is not test-friendly
- ☐ Other (specify)

Test-friendly (Rust) code – FP approach

Pavel Kučera (*1983)

- C++ developer since ~2006
- C#/.NET developer since ~2015
- Fan of FP since ~2018 (F#)
- Fan of Rust since 2023

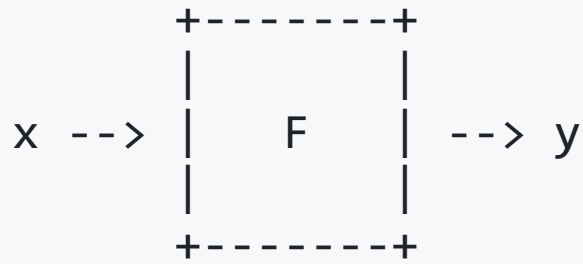


How to make our production code more test-friendly?

We, FP enthusiasts, believe that the key is to make our production code more functional :-)

What does that mean?

What is a Function?



- A "box"
- Single input, Single output
- Every input produces an output (**totality**)
- Same input $\Rightarrow$ same output (**stateless**)
- No side effects (**immutability**)

Quite an limitation! (For good reasons, we (FPs) believe)

Example - PullRequest - naive implementation

```
#[derive(Debug, Clone)]
pub struct PullRequest {
    pub repo: String,
    pub src_branch: String,
    pub dst_branch: String,
    pub state: State,
}
```

Example - PullRequest - naive implementation - State

```
#[derive(Debug, Clone, PartialEq, Eq)]  
pub enum State {  
    New,  
    Open,  
    Approved(Approval),  
    Closed(Reason),  
}
```


Example - PullRequest - naive implementation - Approval

```
#[derive(Debug, Clone, PartialEq, Eq)]  
pub struct Approval {  
    pub approver: String,  
    pub message: String,  
}
```

Example - PullRequest - naive implementation - Reason

```
#[derive(Debug, Clone, PartialEq, Eq)]  
pub struct Reason {  
    pub message: String,  
}
```

Example - PullRequest - naive implementation - impl new

```
impl PullRequest {  
    pub fn new(repo: String, src_branch: String, dst_branch: String) -> Self {  
        PullRequest {  
            repo,  
            src_branch,  
            dst_branch,  
            state: State::New,  
        }  
    }  
}
```

Example - PullRequest - naive implementation - impl open

```
impl PullRequest {  
    pub fn open(&mut self) {  
        self.state = State::Open;  
    }  
}
```

Example - PullRequest - naive implementation - impl approve

```
impl PullRequest {  
    pub fn approve(&mut self, approver: String, message: String) {  
        self.state = State::Approved(Approval { approver, message });  
    }  
}
```

Example - PullRequest - naive implementation - impl close

```
impl PullRequest {  
    pub fn close(&mut self, reason: String) {  
        self.state = State::Closed(Reason { message: reason });  
    }  
}
```

Example - PullRequest - naive implementation - tests - new

```
#[test]
fn test_new_pr_is_in_new_state() {
    let pr = PullRequest::new("repo".to_string(), "src".to_string(), "dst".to_string());
    assert_eq!(pr.state, State::New);
}
```

Example - PullRequest - naive implementation - tests - open

```
#[test]
fn test_pr_open_opens_the_pr() {
    let mut pr = PullRequest::new("repo".to_string(), "src".to_string(), "dst".to_string());
    pr.open();
    assert_eq!(pr.state, State::Open);
}
```


Example - PullRequest - naive implementation - tests - approve

```
#[test]
fn test_pr_approve_approves_the_pr() {
    let mut pr = PullRequest::new("repo".to_string(), "src".to_string(), "dst".to_string());
    pr.approve("Alice".to_string(), "LGTM".to_string());
    assert_eq!(
        pr.state,
        State::Approved(Approval {
            approver: "Alice".to_string(),
            message: "LGTM".to_string()
        })
    );
}
```

Example - PullRequest - naive implementation - tests - close

```
#[test]
fn test_pr_close_closes_the_pr() {
    let mut pr = PullRequest::new("repo".to_string(), "src".to_string(), "dst".to_string());
    pr.close("No longer needed".to_string());
    assert_eq!(
        pr.state,
        State::Closed(Reason {
            message: "No longer needed".to_string()
        })
    );
}
```

running 4 tests

test pr_naive::tests::test_new_pr_is_in_new_state ... ok

test pr_naive::tests::test_pr_close_closes_the_pr ... ok

test pr_naive::tests::test_pr_open_opens_the_pr ... ok

test pr_naive::tests::test_pr_approve_approves_the_pr ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Example - PullRequest - naive implementation - problems?

- ☐ hard to maintain
- ☐ hard to understand
- ☐ hard to reproduce failures
- ☐ Break during refactoring
- ☐ Have lots of duplicated code
- ☐ Create unnecessary stress
- ☐ Other (specify)

Example - PullRequest - naive implementation - problems!

- ☒ hard to maintain
- ☐ hard to understand
- ☒ hard to reproduce failures
- ☐ Break during refactoring
- ☒ Have lots of duplicated code
- ☒ Create unnecessary stress
- ☒ Other (specify)

Example - PullRequest - naive implementation - can we do better?

Example - PullRequest - FP implementation

```
#[derive(Debug, Clone)]
pub struct PullRequestStateMachine;

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Action {
    Open,
    Approve { approver: String, message: String },
    Close { reason: String },
}
```

Example - PullRequest - FP implementation - impl

```
impl PullRequestStateMachine {  
  pub fn apply(state: State, action: Action) -> State {  
    match (state, action) {  
      (State::New, Action::Open) => State::Open,  
      (State::Open, Action::Approve { approver, message }) => {  
        State::Approved(Approval { approver, message })  
      },  
      (State::Open, Action::Close { reason }) => State::Closed(Reason { message: reason }),  
    }  
  }  
}
```


Example - PullRequest - FP implementation - cargo build

```
> cargo build
```

```
error[E0004]: non-exhaustive patterns: `(pr::State::Approved(_), _)` and `(pr::State::Closed(_), _)` not covered
```

```
--> src\pr.rs:31:15
```

```
31 |         match (state, action) {  
    |             ^^^^^^^^^^^^^^^^^ patterns `(pr::State::Approved(_), _)` and `(pr::State::Closed(_), _)` not covered
```

```
= note: the matched value is of type `(pr::State, pr::Action)`
```

```
help: ensure that all possible cases are being handled by adding a match arm with a wildcard pattern, a match arm with multiple or-patterns as shown, or multiple match arms
```

```
36 ~             (State::Open, Action::Close { reason }) => State::Closed(Reason { message: reason }),  
37 ~             (pr::State::Approved(_), _) | (pr::State::Closed(_), _) => todo!(),
```

Example - PullRequest - FP implementation - panic!

```
impl PullRequestStateMachine {  
  /// This may panic!  
  pub fn apply_unchecked(state: State, action: Action) -> State {  
    match (state, action) {  
      (State::New, Action::Open) => State::Open,  
      (State::Open, Action::Approve { approver, message }) => {  
        State::Approved(Approval { approver, message })  
      },  
      (State::Open, Action::Close { reason }) => State::Closed(Reason { message: reason }),  
      (State::Approved(_approval), Action::Close { reason }) => {  
        State::Closed(Reason { message: reason })  
      },  
      (State::Closed(_reason), Action::Open) => State::Open,  
      (state, action) => panic!(  
        "Invalid transition from state {:?} with action {:?}",  
        state, action  
      ),  
    },  
  }  
}
```

Example - PullRequest - FP implementation - error handling

```
#[derive(Error, Debug, Clone, PartialEq)]
pub enum PullRequestStateMachineError {
    #[error("Invalid transition from state {0:?} with action {1:?}")]
    InvalidTransition(State, Action),
}
```

Example - PullRequest - FP implementation - apply - Result

```
impl PullRequestStateMachine {
  pub fn apply(state: &State, action: Action) -> Result<State, PullRequestStateMachineError> {
    match (state, action) {
      (State::New, Action::Open) => Ok(State::Open),
      (State::Open, Action::Approve { approver, message }) => {
        Ok(State::Approved(Approval { approver, message }))
      }
      (State::Open, Action::Close { reason }) => {
        Ok(State::Closed(Reason { message: reason }))
      }
      (State::Approved(_approval), Action::Close { reason }) => {
        Ok(State::Closed(Reason { message: reason }))
      }
      (State::Closed(_reason), Action::Open) => Ok(State::Open),
      (state, action) => Err(PullRequestStateMachineError::InvalidTransition(
        state.clone(),
        action,
      )),
    },
  }
}
```

Example - PullRequest - FP implementation - apply_exhaustive

```
pub fn apply_exhaustive(
    state: &State,
    action: Action,
) -> Result<State, PullRequestStateMachineError> {
    match (state, action) {
        // state: New
        (State::New, Action::Open) => Ok(State::Open),
        (State::New, action @ (Action::Approve { .. } | Action::Close { .. })) => Err(
            PullRequestStateMachineError::InvalidTransition(State::New, action),
        ),

        // state: Open
        (State::Open, Action::Approve { approver, message }) => {
            Ok(State::Approved(Approval { approver, message }))
        }
        (State::Open, Action::Close { reason }) => {
            Ok(State::Closed(Reason { message: reason }))
        }
        (State::Open, action @ Action::Open) => Err(
            PullRequestStateMachineError::InvalidTransition(State::Open, action),
        ),
    }
}
```

```

// state: Approved
(State::Approved(_approval), Action::Close { reason }) => {
    Ok(State::Closed(Reason { message: reason }))
}
(State::Approved(approval), action @ (Action::Open | Action::Approve { .. }) ) => {
    Err(PullRequestStateMachineError::InvalidTransition(
        State::Approved(approval.clone()),
        action,
    ))
}

// state: Closed
(State::Closed(_reason), Action::Open) => Ok(State::Open),
(State::Closed(reason), action @ (Action::Approve { .. } | Action::Close { .. }) ) => {
    Err(PullRequestStateMachineError::InvalidTransition(
        State::Closed(reason.clone()),
        action,
    ))
}
}
}

```

Example - PullRequest - FP implementation - PullRequest API

```
impl PullRequest {  
  
    pub fn open(&mut self) {  
        self.apply(Action::Open);  
    }  
  
    pub fn approve(&mut self, approver: String, message: String) {  
        self.apply(Action::Approve { approver, message });  
    }  
  
    pub fn close(&mut self, reason: String) {  
        self.apply(Action::Close { reason });  
    }  
}
```

Example - PullRequest - FP implementation - PullRequest apply

```
impl PullRequest {  
    fn apply(&mut self, action: Action) {  
        debug!("Applying action {:?} in state {:?}", action, self.state);  
  
        match PullRequestStateMachine::apply(&self.state, action) {  
            Ok(new_state) => self.state = new_state,  
            Err(e) => {  
                error!(  
                    "Error applying action: {}, keeping state {:?}",  
                    e, self.state  
                );  
            }  
        }  
    }  
}
```


Example - PullRequest - FP implementation - tests

```
#[cfg(test)]  
mod tests {  
  
    ???  
  
}
```

Why do we bother with tests?

- Find regressions sooner ("shift left")
- Be more confident when refactoring
- Help us develop new functionality
- Examples how to use API of our code
- Other (specify)

Why do I bother with tests?

They forces me to think about the production code

- Edge cases
- Error handling
- API

Why to bother with tests of PullRequestStateMachine?

- ✓ Edge cases (enforced by the compiler)
- ✓ Error handling (enforced by the compiler)
- ✓ API (used by `fn PullRequest::apply`)

How would the tests look like anyway?

- *parametrized tests* covering all the combinations of states and actions => Which is what we already do in the production code (in `apply_exhaustive`)!
- The impure (mutable) part of the code (`fn PullRequest::apply`) – just a thin wrapper around the pure (functional) part, so there is not much to test there.

Conclusion

The best tests are those that we don't need to write!

FP + Rust can help us to write code that is easy to reason about even without tests.

TLDR

Separate the logic into pure functions.

Tip - Parametrized tests

Use `rstest` when the test shape is the same, but the inputs change.

```
use rstest::rstest;

#[rstest]
#[case(State::New, true)]
#[case(State::Open, false)]
#[case(State::Approved(Approval { approver: "Alice".into(), message: "LGTM".into() }), false)]
#[case(State::Closed(Reason { message: "Merged".into() }), false)]
fn open_is_allowed(#[case] state: State, #[case] expected: bool) {
    let actual = PullRequestStateMachine::apply(&state, Action::Open)
        .is_ok();

    assert_eq!(expected, actual);
}
```

Tip - Fixtures

Use `rstest` fixtures

for common setup. Keep only the interesting variation in the test.

```
#[fixture]
fn pr() -> PullRequest {
    PullRequest::new("repo".into(), "topic".into(), "main".into())
}

#[rstest]
fn new_pr_is_new(pr: PullRequest) {
    assert_eq!(State::New, pr.state);
}
```

Tip - Expensive setup

Use `#[once]` for read-only fixtures that are expensive to create.

```
#[fixture]
#[once]
fn repo() -> Repository {
    Repository::clone_from("git@github.com:tencek/talks.git")
}

#[rstest]
fn new_pr_targets_main(repo: &Repository) {
    let pr = repo.open_pr("feature");

    assert_eq!("main", pr.dst_branch);
}
```


Tip - Assert floats?

What is wrong with this test?

```
#[test]
fn calculates_approval_ratio() {
    let approved = 1.0_f64;
    let total = 10.0_f64;

    assert_eq!(0.1, approved / total);
}
```

Why?

Tip - Assert floats

Use `approx` instead of `assert_eq!` for floating point values.

```
use approx::{assert_abs_diff_eq, assert_relative_eq};

let ratio = approval_ratio(1, 10);
assert_abs_diff_eq!(0.1, ratio, epsilon = 1e-9);

let score = reviewer_confidence(&pull_request);
assert_relative_eq!(0.95, score, max_relative = 1e-6);
```

- absolute diff: constant magnitude, e.g 0.0 - 1.0
- relative diff: good when magnitude changes

Tip - Domain floats

For new-types, implement `AbsDiffEq` / `RelativeEq` once.

```
[derive(Debug, Copy, Clone, PartialEq)]
struct ApprovalRatio(f64);

impl approx::AbsDiffEq for ApprovalRatio {
    type Epsilon = f64;

    fn default_epsilon() -> f64 { 1e-9 }

    fn abs_diff_eq(&self, other: &Self, epsilon: f64) -> bool {
        self.0.abs_diff_eq(&other.0, epsilon)
    }
}
```

Tip - Logs in tests - code

Put the context in production code, e.g. with `#[instrument]` .

```
use tracing::{debug, instrument};

#[instrument(skip(self), fields(repo = %self.repo, state = ?self.state))]
fn apply(&mut self, action: Action) {
    debug!(?action, "applying PR action");

    match PullRequestStateMachine::apply(&self.state, action) {
        Ok(new_state) => self.state = new_state,
        Err(error) => tracing::warn!(%error, "keeping current state"),
    }
}
```

Tip - Logs in tests - test

Use `test-log` to see logs emitted by the production code.

```
#[test_log::test]
fn cannot_approve_new_pr() {
    let mut pr = PullRequest::new("talks".into(), "topic".into(), "main".into());

    pr.approve("Alice".into(), "LGTM".into());

    assert_eq!(State::New, pr.state);
}
```

```
$env:RUST_LOG = "trace"
cargo test cannot_approve_new_pr -- --show-output
```

With `rstest`: `#[test_log::test(rstest)]`

Tip - Property-based tests

Use `proptest` for property-based testing.

It allows to test that certain properties of your code hold for arbitrary inputs, and if a failure is found, automatically finds the minimal test case to reproduce the problem.

```
use proptest::prelude::*;
proptest! {
    #[test]
    fn pr_new_never_crashes(repo in "{1,80}", src in "{1,80}", dst in "{1,80}") {
        let _ = PullRequest::new(repo.clone(), src.clone(), dst.clone());
    }
}
```

I but a subtle bug in `PullRequest::new` logging that causes it to panic when the repo name is empty or weird.

Will proptest find it? Let's see!

Tip - Documentation tests

Use [rustdoc doctests](#)

for examples that are part of the public contract.

```
/// Opens a new pull request.  
///  
/// ```  
/// use my_crate::PullRequest;  
///  
/// let mut pr = PullRequest::new("repo".into(), "src".into(), "main".into());  
/// pr.open();  
/// assert!(pr.is_open());  
/// ```  
pub fn open(&mut self) {  
    // ...  
}
```

Tip - Integration tests

Cargo treats each `tests/*.rs` file as a separate crate.

```
// tests/pull_request.rs
use my_crate::{Action, PullRequestStateMachine, State};

#[test]
fn opening_new_pr() {
    let actual = PullRequestStateMachine::apply(&State::New, Action::Open);
    assert_eq!(Ok(State::Open), actual);
}
```

- tests public API behavior
- good place for end-to-end scenarios
- use a `test_support` crate for shared helpers

Tip - Data-driven tests

Use data files when expected results come from an external source.

- keep source data readable in `assets/`
- parse into typed structs with `serde_json`
- use `include_str!` for small deterministic files